SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION **COURSE CODE:** ITA05

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

2. OCR Pipeline for Document Processing

A document processing system must:

1. Read input images containing printed text
2. Enhance image quality (noise removal, contrast improvement)
3. Segment text regions
4. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

Answer:

```
FUNCTION process_single_document(image_path):
    # 1. Preprocessing Stage (Noise reduction & contrast enhancement)
    img = cv2.imread(image_path)
    IF img IS NULL: RETURN NULL
    gray = cv2.cvtColor(img, COLOR_BGR2GRAY)

    # Remove high-frequency noise while preserving key edge boundaries
    denoised = cv2.fastNlMeansDenoising(gray, h=10)

    # Adaptive thresholding handles non-uniform illumination and local low contrast
    enhanced = cv2.adaptiveThreshold(denoised, 255, ADAPTIVE_THRESH_GAUSSIAN_C,
                                     THRESH_BINARY, 11, 2)

    # 2. Segmentation Stage (Isolating text lines/regions)
    # Horizontal rectangular kernel for line/character grouping
    kernel = cv2.getStructuringElement(MORPH_RECT, (15, 3))
    morphed = cv2.morphologyEx(enhanced, MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(morphed, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)

    # Filter contours by aspect ratio and minimum area to remove residual artifacts
    text_regions = []
    annotated_img = img.copy()

    FOR EACH cnt IN contours:
        x, y, w, h = cv2.boundingRect(cnt)
        IF w > 20 AND h > 10 AND (w / FLOAT(h)) > 0.5:
            text_regions.APPEND(img[y:y+h, x:x+w])
            cv2.rectangle(annotated_img, (x, y), (x+w, y+h), (0, 255, 0), 2)
```

```
    # 3. Extraction Stage (OCR Integration)
    extracted_text = ""
    FOR EACH region IN text_regions:
        text = pytesseract.image_to_string(region, config='--psm 6')
        extracted_text += text + "
"

    RETURN extracted_text, annotated_img

FUNCTION batch_ocr_pipeline(image_folder_path):
    image_files = get_all_images(image_folder_path)
    FOR EACH file IN image_files:
        text, annotated_img = process_single_document(file)
        IF text IS NOT NULL:
            DISPLAY annotated_img
            SAVE text TO file + "_ocr.txt"
```

6. Real-Time Object Detection Pipeline

A system must:

1. Capture video frames
2. Detect objects using a trained model
3. Draw bounding boxes and labels
4. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

Answer:

```
FUNCTION run_realtime_detection():
    # Load lightweight deep learning network optimized for real-time inference
    net = cv2.dnn.readNetFromONNX("yolov8n.onnx")
    net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
    net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)  # Or DNN_TARGET_CUDA for GPU
acceleration

    cap = cv2.VideoCapture(0)  # Live video camera input stream

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        # Preprocessing: Convert image to normalized 4D tensor blob
        blob = cv2.dnn.blobFromImage(frame, 1/255.0, (416, 416), swapRB=True, crop=False)
        net.setInput(blob)

        # Forward pass through model
        outputs = net.forward()
        boxes, confidences, class_ids = parse_detections(outputs, confidence_threshold=0.5)

        # Non-Maximum Suppression (NMS) to eliminate duplicate overlapping bounding boxes
        indices = cv2.dnn.NMSBoxes(boxes, confidences, score_threshold=0.5,
nms_threshold=0.4)

        # Visualization and annotation
        FOR EACH i IN indices:
            box = boxes[i]
```

```
            x, y, w, h = box[0], box[1], box[2], box[3]
            label = CLASSES[class_ids[i]] + ": " + STR(ROUND(confidences[i], 2))
            cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 0, 0), 2)
            cv2.putText(frame, label, (x, y-10), FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

        cv2.imshow("Real-Time Object Detector", frame)
        IF cv2.waitKey(1) == ORD('q'): BREAK

    cap.release()
    cv2.destroyAllWindows()
```

5. Facial Expression Recognition System

A mobile-based system must:

1. Capture images from a camera
2. Detect faces
3. Extract facial features
4. Classify expressions (happy, sad, neutral)
5. Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

Answer:

```
FUNCTION facial_expression_pipeline(camera_index=0):
    cap = cv2.VideoCapture(camera_index)
    # Lightweight Haar Cascade detector for mobile real-time performance
    face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_alt2.xml")

    # Load quantized MobileNet/Mini-Xception lightweight classifier
    expression_net = cv2.dnn.readNetFromTFLite("expression_classifier_quant.tflite")
    LABELS = ["Happy", "Sad", "Neutral"]

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        gray = cv2.cvtColor(frame, COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, scaleFactor=1.2, minNeighbors=4,
minSize=(40, 40))

        FOR EACH (x, y, w, h) IN faces:
            # Extract Face Region of Interest (ROI)
            face_roi = gray[y:y+h, x:x+w]
            face_resized = cv2.resize(face_roi, (48, 48))

            # Preprocess ROI for classification tensor
            blob = cv2.dnn.blobFromImage(face_resized, 1.0/255.0, (48, 48), (0, 0, 0),
swapRB=False)
            expression_net.setInput(blob)

            # Efficient classification pass
            predictions = expression_net.forward()
            class_idx = ARGMAX(predictions[0])
            expression_label = LABELS[class_idx]
            confidence = predictions[0][class_idx]
```

```
            # Annotation rendering
            display_text = expression_label + " (" + STR(ROUND(confidence * 100, 1)) + "%)"
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 255), 2)
            cv2.putText(frame, display_text, (x, y-10), FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
255), 2)

        cv2.imshow("Facial Expression Monitor", frame)
        IF cv2.waitKey(1) == ORD('q'): BREAK

    cap.release()
    cv2.destroyAllWindows()
```

12. Face Detection with ROI Extraction

A system must:

1. Detect faces in an image
2. Extract face regions
3. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

Answer:

```
FUNCTION detect_and_save_faces(image_path, output_dir):
    img = cv2.imread(image_path)
    IF img IS NULL: RETURN NULL, 0
    gray = cv2.cvtColor(img, COLOR_BGR2GRAY)

    # Use Haar Cascade Face Detector with specific scaling to prevent duplicate overlapping
ROIs
    face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

    # ScaleFactor and minNeighbors control detection robustness and redundancy filtering
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    face_count = 0
    FOR EACH (x, y, w, h) IN faces:
        # Extract Region of Interest (ROI)
        face_roi = img[y:y+h, x:x+w]

        # Save cropped faces to filesystem with unique identifiers
        save_path = output_dir + "/face_" + STR(face_count) + ".jpg"
        cv2.imwrite(save_path, face_roi)

        # Draw bounding boxes on primary visualization frame
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
        face_count += 1

    RETURN img, face_count
```

10. Edge Detection Pipeline for Real-Time Video

A system must:

1. Capture video frames
2. Apply edge detection
3. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

Answer:

```
FUNCTION run_realtime_edge_detection(video_source=0):
    cap = cv2.VideoCapture(video_source)

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        # Stage 1: Preprocessing Module (Grayscale conversion & Gaussian smoothing)
        gray = cv2.cvtColor(frame, COLOR_BGR2GRAY)

        # 5x5 Gaussian Kernel attenuates high-frequency sensor noise while preserving major
edge contours
        blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)

        # Stage 2: Edge Detection Module (Canny Edge Detection with dynamic/adaptive
hysteresis thresholds)
        median_val = MEDIAN(blurred)
        lower_thresh = INT(MAX(0, (1.0 - 0.33) * median_val))
        upper_thresh = INT(MIN(255, (1.0 + 0.33) * median_val))

        edges = cv2.Canny(blurred, threshold1=lower_thresh, threshold2=upper_thresh)

        # Stage 3: Visualization Module
        # Convert 1-channel edge map back to 3-channel BGR for unified side-by-side display
        edges_bgr = cv2.cvtColor(edges, COLOR_GRAY2BGR)
        combined_display = CONCAT_HORIZONTAL(frame, edges_bgr)

        cv2.imshow("Real-Time Edge Detection Feed (Original vs Canny Edges)",
combined_display)
        IF cv2.waitKey(1) == ORD('q'): BREAK

    cap.release()
    cv2.destroyAllWindows()
```

Code of Conduct:

I __DHARSHINI S (192521398)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient